포팅 매뉴얼

날짜	2026년 5월 19일
분류	

포팅 매뉴얼 (DrawRun 프로젝트)

SSAFY 14기 - S14P31A103

배포 서버: k14a103.p.ssafy.io

1. 시스템 개요

항목	내용
프로젝트명	DrawRun (드로우런)
배포 서버	EC2 (Ubuntu 24.04) - 172.26.9.189
배포 방식	Docker Compose (단일 서버, 컨테이너 9개)
프론트엔드	Flutter Android 앱 (APK)
백엔드	Spring Boot (Java 21)
데이터베이스	PostgreSQL + PostGIS, Redis
SSL	Certbot (Let's Encrypt)
CI/CD	GitLab MR → Jenkins 자동 빌드/배포
알림	WebhookBridge → Mattermost
모니터링	Promtail → Loki → Grafana

2. 기술 스택 및 버전

2.1. Docker 컨테이너 구성

서비스	컨테이너명	이미지	포트	역할
Nginx	nginx	nginx:latest	80, 443	리버스 프록시, SSL 종단
Spring Boot	backend	infra-backend	8080 (내부)	REST API 서버
Jenkins	jenkins	jenkins/jenkins:lts	8080 (내부)	CI/CD 파이프라인
WebhookBridge	webhookbridge	infra-webhookbridge	9103 (내부)	GitLab·Jenkins 알림 중계
PostgreSQL	postgres	postgis/postgis:16-3.4	5432	지리 데이터 포함 메인 DB
Redis	redis	redis:alpine	6379	JWT/랭킹 캐시
Grafana	grafana	grafana/grafana:10.4.0	3000 (내부)	로그 대시보드

서비스	컨테이너명	이미지	포트	역할
Loki	loki	grafana/loki:2.9.0	3100 (내부)	로그 저장 (30일)
Promtail	promtail	grafana/promtail:2.9.0	-	Docker 로그 수집

2.2. 프론트엔드 (Flutter App)

항목	버전/내용
Flutter SDK	3.41.7
언어	Dart
상태관리	Riverpod (flutter_riverpod)
라우팅	GoRouter
HTTP	Dio + JWT 인터셉터
지도	flutter_naver_map
GPS	geolocator
센서	pedometer, sensors_plus
빌드 결과물	APK (Android)

2.3. 외부 API

서비스	용도	환경변수
기상청 Open API	러닝 중 현재 위치 날씨 조회	WEATHER_API_KEY
네이버 지도 API	Flutter 지도 렌더링	NAVER_MAP_CLIENT_ID

---

3. 포트 매핑 테이블

서비스	호스트 포트 → 컨테이너 포트	외부 접근
nginx	80 → 80, 443 → 443	O (공개)
backend	8080 (내부만)	X (Nginx /api/* 경유)
jenkins	8080 (내부만)	X (Nginx /jenkins 경유)
webhookbridge	9103 (내부만)	X (Nginx /infra/webhook 경유)
postgres	5432 → 5432	O (공개)
redis	6379 → 6379	O (공개)
grafana	3000 (내부만)	X (Nginx /grafana/ 경유)
loki	3100 (내부만)	X (내부 통신만)
promtail	-	X (내부 통신만)

3.1. Nginx 라우팅 규칙

경로	프록시 대상	설명
/api/*	backend:8080	Spring Boot REST API

경로	프록시 대상	설명
/jenkins	jenkins:8080	Jenkins CI/CD 콘솔
/grafana/	grafana:3000	로그 모니터링 대시보드
/infra/webhook	webhookbridge:9103	GitLab/Jenkins 웹훅 수신
/uploads/tracks/*	EC2 로컬 파일	러닝 경로 이미지 직접 서빙
/uploads/challenge/*	EC2 로컬 파일	챌린지 폴리곤 이미지 직접 서빙

4. 사전 준비 사항

4.1. EC2 서버 요구사항

- OS: Ubuntu 24.04 LTS
- Docker, Docker Compose 설치 완료
- GitLab 레포지토리 접근 권한 (PAT 발급 필요)
- 기상청 Open API 키 발급
- 네이버 지도 API 클라이언트 ID 발급
- Mattermost Incoming Webhook URL 발급
- 도메인 연결: k14a103.p.ssafy.io

4.2. uploads 디렉토리 생성

backend 컨테이너가 이미지 파일을 EC2 호스트에 volume mount로 저장하므로 사전에 디렉토리를 생성해야 합니다.

```
mkdir -p /home/ubuntu/uploads/tracks
mkdir -p /home/ubuntu/uploads/challenge
sudo chown -R ubuntu:ubuntu /home/ubuntu/uploads
```

4.3. 방화벽 설정

```
sudo ufw allow 22/tcp    # SSH
sudo ufw allow 80/tcp    # HTTP
sudo ufw allow 443/tcp   # HTTPS
sudo ufw reload
sudo ufw status
```

5. SSL 인증서 발급 (Certbot)

certbot 컨테이너로 SSL 인증서를 발급합니다. 호스트에 certbot을 별도 설치할 필요 없습니다.

5.1. 디렉토리 생성

```
mkdir -p /home/ubuntu/certbot/conf
mkdir -p /home/ubuntu/certbot/www
```

5.2. `nginx.conf` 443 블록 임시 주석처리 후 Nginx 실행

```
docker compose up -d nginx
```

5.3. certbot 컨테이너로 인증서 발급

```
docker run --rm \
  -v /home/ubuntu/certbot/conf:/etc/letsencrypt \
  -v /home/ubuntu/certbot/www:/var/www/certbot \
  certbot/certbot certonly --webroot \
  --webroot-path=/var/www/certbot \
  --email <이메일> \
  --agree-tos --no-eff-email \
  -d k14a103.p.ssafy.io
```

5.4. `nginx.conf` 443 블록 주석 해제 후 재시작

```
docker compose restart nginx
```

> 컨테이너 내부 경로: `/etc/letsencrypt/live/k14a103.p.ssafy.io/` (nginx.conf에 명시된 경로)
> EC2 호스트 실제 경로: `/home/ubuntu/certbot/conf/live/k14a103.p.ssafy.io/`
>
> 현재 만료일: 2026-07-22

---

6. Docker Compose 실행

6.1. 레포지토리 클론

```
cd /home/ubuntu
git clone https://lab.ssafy.com/s14p31a103/S14P31A103.git
cd S14P31A103/infra
```

6.2. 전체 서비스 기동

```
# 전체 컨테이너 기동
docker compose up -d

# 기동 확인
docker ps
```

정상 기동 시 아래 9개 컨테이너가 모두 `Up` 상태여야 합니다.

컨테이너명	확인 방법
nginx	0.0.0.0:80, 0.0.0.0:443 바인딩 확인
backend	Up (8080/tcp)
jenkins	Up (8080/tcp, 50000/tcp)
webhookbridge	Up (9103/tcp)
postgres	0.0.0.0:5432 바인딩 확인
redis	0.0.0.0:6379 바인딩 확인
grafana	Up (3000/tcp)
loki	Up (3100/tcp)
promtail	Up

6.3. 개별 서비스 재시작

```
# 백엔드만 재빌드 후 재시작
docker compose up -d --build backend
```

---

7. Jenkins 초기 설정

7.1. 초기 관리자 비밀번호 확인

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPasswor
d
```

1. 브라우저에서 `https://k14a103.p.ssafy.io/jenkins` 접속
2. 초기 관리자 비밀번호 입력
3. **Install suggested plugins** 선택
4. 관리자 계정 생성 (Create First Admin User)
   - 아이디, 비밀번호, 이름, 이메일 설정
5. Jenkins 사용 시작

7.2. 필수 플러그인 설치

Manage Jenkins → **Plugins** → **Available plugins** 에서 검색 후 설치:

- GitLab Plugin
- Docker Pipeline
- Pipeline

7.3. Jenkins Credentials 등록

Manage Jenkins → **Credentials** → **(global)** → **Add Credentials** 에서 아래 두 가지를 등록합니다.

1. `BACKEND_ENV`
   - Kind: Secret File
   - File: 로컬에서 작성한 `.env` 파일 업로드
   - ID: BACKEND_ENV
2. `gitlab-pat`
   - Kind: Username with password
   - Username: 아무거나
   - Password: GitLab PAT
   - ID: gitlab-pat

7.4. Jenkins Pipeline Job 생성

1. **New Item** → **Pipeline** → 이름: `drawrun-backend`
2. **Build Triggers** → `Build when a change is pushed to GitLab` 체크
3. **Pipeline** 설정:
   - Definition: `Pipeline script from SCM`
   - SCM: `Git`
   - Repository URL: `https://lab.ssafy.com/s14p31a103/S14P31A103.git`
   - Credentials: `gitlab-pat`
   - Branch Specifier: `*/develop`
   - Script Path: `Jenkinsfile`

---

8. 환경변수 설정 (.env)

8.1. `.env` 파일 변수명 및 설명

분류	변수명	설명
Mattermost 알림 설정	NOTIFICATION_WEBHOOK_URL_GITLAB	GitLab 이벤트용 Mattermost Incoming Webhook URL

분류	변수명	설명
	NOTIFICATION_CHANNEL_GITLAB	GitLab 알림 채널명 (예: a103-gitlab)
	NOTIFICATION_WEBHOOK_URL_JENKINS	Jenkins 빌드 결과용 Mattermost Incoming Webhook URL
	NOTIFICATION_CHANNEL_JENKINS	Jenkins 알림 채널명 (예: a103-jenkins)
	NOTIFICATION_INTERNAL_URL	WebhookBridge 내부 접근 URL (예: http://k14a103.p.ssafy.io:9090)
	NOTIFICATION_PUBLIC_URL	Jenkins 외부 접근 URL (예: https://k14a103.p.ssafy.io/jenkins)
	USER_MAPPING_JSON	GitLab 유저명 → Mattermost 유저명 매핑 JSON
	LABEL_MAPPING_JSON	MR 라벨 → 담당자 매핑 JSON
데이터베이스	DB_URL	JDBC URL (예: jdbc:postgresql://postgres:5432/running)
	DB_HOST	PostgreSQL 호스트 (Docker 내부: postgres)
	DB_USERNAME	DB 사용자명
	DB_PASSWORD	DB 비밀번호
	DB_NAME	DB 이름 (예: running)
Redis	REDIS_HOST	Redis 호스트 (Docker 내부: redis)
	REDIS_PORT	Redis 포트 (기본: 6379)
	REDIS_PASSWORD	Redis 비밀번호 (미설정 시 빈 값)
애플리케이션	SPRING_PROFILES_ACTIVE	Spring 프로파일 (dev / prod)
	JWTSECRET	JWT 서명 시크릿 키
	GRAFANA_PASSWORD	Grafana 관리자 비밀번호
외부 API	WEATHER_API_KEY	기상청 Open API 인증키
	GMS_KEY	GMS API 키
	NAVER_SEARCH_CLIENT_ID	네이버 검색 API Client ID
	NAVER_SEARCH_CLIENT_SECRET	네이버 검색 API Client Secret
OSM 도로 데이터 (Google Drive)	GDRIVE_WAYS_URL	OSM ways 데이터 Google Drive URL
	GDRIVE_CROSSINGS_URL	OSM crossings 데이터 Google Drive URL
	GDRIVE_DISTRICTS_URL	OSM districts 데이터 Google Drive URL

8.2. `.env` 파일 생성

아래의 예시를 참고해 `.env` 파일을 로컬에서 작성합니다.

```
#.env 파일 예시
# Mattermost
NOTIFICATION_WEBHOOK_URL_GITLAB=https://meeting.ssafy.com/hooks/<your_hook>
```

```
NOTIFICATION_CHANNEL_GITLAB=a103-gitlab
NOTIFICATION_WEBHOOK_URL_JENKINS=https://meeting.ssafy.com/hooks/<you
r_hook>
NOTIFICATION_CHANNEL_JENKINS=a103-jenkins
NOTIFICATION_INTERNAL_URL=http://k14a103.p.ssafy.io:9090
NOTIFICATION_PUBLIC_URL=https://k14a103.p.ssafy.io/jenkins
USER_MAPPING_JSON={"이름":"mattermost_id"}
LABEL_MAPPING_JSON={"BE":["이름1","이름2"]}

# Database
DB_URL=jdbc:postgresql://postgres:5432/running
DB_HOST=postgres
DB_USERNAME=root
DB_PASSWORD=<비밀번호>
DB_NAME=running

# Redis
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_PASSWORD=

# App
SPRING_PROFILES_ACTIVE=dev
JWTSECRET=<JWT_시크릿>
GRAFANA_PASSWORD=<Grafana_비밀번호>

# External API
WEATHER_API_KEY=<기상청_API_KEY>
GMS_KEY=<GMS_KEY>
NAVER_SEARCH_CLIENT_ID=<네이버_검색_CLIENT_ID>
NAVER_SEARCH_CLIENT_SECRET=<네이버_검색_CLIENT_SECRET>

# OSM Data (Google Drive)
GDRIVE_WAYS_URL=https://drive.google.com/uc?id=<file_id>
GDRIVE_CROSSINGS_URL=https://drive.google.com/uc?id=<file_id>
GDRIVE_DISTRICTS_URL=https://drive.google.com/uc?id=<file_id>
```

---

9. GitLab Webhook 설정

GitLab Repository → Settings → Webhooks → Add new webhook

항목	값
URL (Jenkins)	`https://k14a103.p.ssafy.io/jenkins/project/drawrun-backend`

항목	값
URL (WebhookBridge)	`https://k14a103.p.ssafy.io/infra/webhook`
Trigger	Merge request events, Push events
SSL verification	활성화

10. 주요 접속 URL

서비스	URL	비고
API 서버	https://k14a103.p.ssafy.io/api/	Spring Boot REST API
Jenkins	https://k14a103.p.ssafy.io/jenkins	CI/CD 콘솔
Grafana	https://k14a103.p.ssafy.io/grafana/	로그 모니터링
러닝 경로 이미지	https://k14a103.p.ssafy.io/uploads/tracks/{파일명}	Nginx 직접 서빙
챌린지 이미지	https://k14a103.p.ssafy.io/uploads/challenge/{파일명}	Nginx 직접 서빙

11. 배포 흐름 (CI/CD)

단계	내용
1. MR 생성	개발자가 GitLab에 Merge Request 생성
2. Webhook 트리거	GitLab이 Jenkins와 WebhookBridge에 동시 Webhook 전송
3. Mattermost 알림	WebhookBridge가 팀 채널에 MR 생성 알림 전송
4. Jenkins 빌드 시작	Init: 빌드 시작 시간 기록, WebhookBridge에 STARTED 알림
5. Prepare Env	Jenkins Credentials에서 infra/.env 주입
6. Build	./gradlew bootJar -x test (백엔드만 해당, Flutter는 수동 빌드)
7. Deploy	`docker compose up --build` (해당 컨테이너만 재시작)
8. 결과 알림	SUCCESS/FAILURE 결과를 WebhookBridge 통해 Mattermost 전송

12. 트러블슈팅

12.1. 컨테이너 상태 확인

```
# 전체 컨테이너 상태
docker ps

# 특정 컨테이너 로그 확인
docker logs backend --tail 100
docker logs jenkins --tail 100
docker logs nginx --tail 50
```

12.2. Nginx 설정 오류

```
docker exec nginx nginx -t
docker restart nginx
```

12.3. SSL 인증서 오류

```
# 인증서 수동 갱신
docker run --rm \
    -v /home/ubuntu/certbot/conf:/etc/letsencrypt \
    -v /home/ubuntu/certbot/www:/var/www/certbot \
    certbot/certbot renew

# 갱신 후 Nginx 재시작
docker compose restart nginx
```

12.4. PostgreSQL 접속 확인

```
docker exec -it postgres psql -U root -d running
```

12.5. Redis 접속 확인

```
docker exec -it redis redis-cli ping
# 응답: PONG
```

12.6. uploads 디렉토리 권한 오류

```
sudo chown -R ubuntu:ubuntu /home/ubuntu/uploads
sudo chmod -R 755 /home/ubuntu/uploads
```

12.7. 디스크 공간 부족

```
df -h
docker image prune -f
docker container prune -f
docker volume prune -f
```